Chapter 17: Reasoning Techniques

第十七章：推理技术

This chapter delves into advanced reasoning methodologies for intelligent agents, focusing on multi-step logical inferences and problem-solving. These techniques go beyond simple sequential operations, making the agent's internal reasoning explicit. This allows agents to break down problems, consider intermediate steps, and reach more robust and accurate conclusions. A core principle among these advanced methods is the allocation of increased computational resources during inference. This means granting the agent, or the underlying LLM, more processing time or steps to process a query and generate a response. Rather than a quick, single pass, the agent can engage in iterative refinement, explore multiple solution paths, or utilize external tools. This extended processing time during inference often significantly enhances accuracy, coherence, and robustness, especially for complex problems requiring deeper analysis and deliberation.

本章深入探讨面向智能体的高级推理方法，侧重多步逻辑推断与问题求解。这类技术超越简单的顺序处理，把智能体的内部推理外显出来，便于拆解问题、权衡中间步骤并得到更稳健、准确的结论。这些方法的一个共同核心，是在推理阶段投入更多计算资源：为智能体或其底层 LLM 分配更长的处理时间或更多推理步数，再生成回答。与「一口气答完」相比，智能体可以迭代改进、探索多条路径或调用外部工具。拉长推理期的计算往往能明显提高准确性、连贯性与稳健性，对需要深入分析与反复斟酌的复杂问题尤其有效。

Practical Applications & Use Cases

实际应用与用例

Practical applications include:

实际应用包括：

- **Complex Question Answering:** Facilitating the resolution of multi-hop queries, which necessitate the integration of data from diverse sources and the execution of logical deductions, potentially involving the examination of multiple reasoning paths, and benefiting from extended inference time to synthesize information.
- **Mathematical Problem Solving:** Enabling the division of mathematical problems into smaller, solvable components, illustrating the step-by-step process, and employing code execution for precise computations, where prolonged inference enables more intricate code generation and validation.
- **Code Debugging and Generation:** Supporting an agent's explanation of its rationale for generating or correcting code, pinpointing potential issues sequentially, and iteratively refining the code based on test results (Self-Correction), leveraging extended inference time for thorough debugging cycles.
- **Strategic Planning:** Assisting in the development of comprehensive plans through reasoning across various options, consequences, and preconditions, and adjusting plans based on real-time feedback (ReAct), where extended deliberation can lead to more effective and reliable plans.
- **Medical Diagnosis:** Aiding an agent in systematically assessing symptoms, test outcomes, and patient histories to reach a diagnosis, articulating its reasoning at each phase, and potentially utilizing external instruments for data retrieval (ReAct). Increased inference time allows for a more comprehensive differential diagnosis.
- **Legal Analysis:** Supporting the analysis of legal documents and precedents to formulate arguments or provide guidance, detailing the logical steps taken, and ensuring logical consistency through self-correction. Increased inference time allows for more in-depth legal research and argument construction.

- 复杂问答： 求解多跳查询，需整合多源数据并做逻辑推断，可能要比较多条推理路径；更长的推理时间有助于把信息综合成可靠答案。
- 数学解题： 把大题拆成小题、展示逐步推导，并用代码执行保证数值精确；延长推理有利于生成更复杂的代码并完成校验。
- 代码调试与生成： 让智能体说清为何生成或修改代码、按序排查问题，并据测试结果迭代改进（自我纠正）；更充分的推理时间可支撑完整的调试闭环。
- 战略规划： 在多种选项、后果与前提之间推理以拟定周全方案，并按实时反馈调整（ReAct）；更长的斟酌往往带来更有效、更稳妥的计划。
- 医学诊断： 系统梳理症状、检查与病史以形成判断，在各阶段说明推理链条，并可配合外部工具检索（ReAct）；更长推理有助于做更完整的鉴别诊断。
- 法律分析： 梳理法规与判例以构建论证或给出指引，逐步展示逻辑，并用自我纠正保持一致性；更长推理便于开展更深入的研究与论证。

Reasoning techniques

推理技术

To start, let's delve into the core reasoning techniques used to enhance the problem-solving abilities of AI models..

下文先介绍用于增强 AI 模型解题能力的几项核心推理技术。

Chain-of-Thought (CoT) prompting significantly enhances LLMs complex reasoning abilities by mimicking a step-by-step thought process (see Fig. 1). Instead of providing a direct answer, CoT prompts guide the model to generate a sequence of intermediate reasoning steps. This explicit breakdown allows LLMs to tackle complex problems by decomposing them into smaller, more manageable sub-problems. This technique markedly improves the model's performance on tasks requiring multi-step reasoning, such as arithmetic, common sense reasoning, and symbolic manipulation. A primary advantage of CoT is its ability to transform a difficult, single-step problem into a series of simpler steps, thereby increasing the transparency of the LLM's reasoning process. This approach not only boosts accuracy but also offers valuable insights into the model's decision-making, aiding in debugging and comprehension. CoT can be implemented using various strategies, including offering few-shot examples that demonstrate step-by-step reasoning or simply instructing the model to "think step by step." Its effectiveness stems from its ability to guide the model's internal processing toward a more deliberate and logical progression. As a result, Chain-of-Thought has become a cornerstone technique for enabling advanced reasoning capabilities in contemporary LLMs. This enhanced transparency and breakdown of complex problems into manageable sub-problems is particularly important for autonomous agents, as it enables them to perform more reliable and auditable actions in complex environments.

思维链（**Chain-of-Thought, CoT**） 提示通过模仿人类逐步思考，显著增强 LLM 的复杂推理能力（见图 1）。CoT 不直接给答案，而是引导模型产出中间推理步骤，把难题拆成更小、更好处理的子问题，从而在算术、常识推理、符号操作等多步任务上表现更好。其主要长处之一，是把「一步难算」变成「多步可解」，提高推理透明度，既利于准确率，也便于理解决策与排错。实现上可提供 few-shot 逐步示范，或简单要求「一步一步想」。有效之处在于把模型的内部处理推向更有意识、更有逻辑的推进方式。思维链已是当代 LLM 高级推理的基石；对自主智能体尤其关键，能在复杂环境里带来更可靠、更可审计的行为。

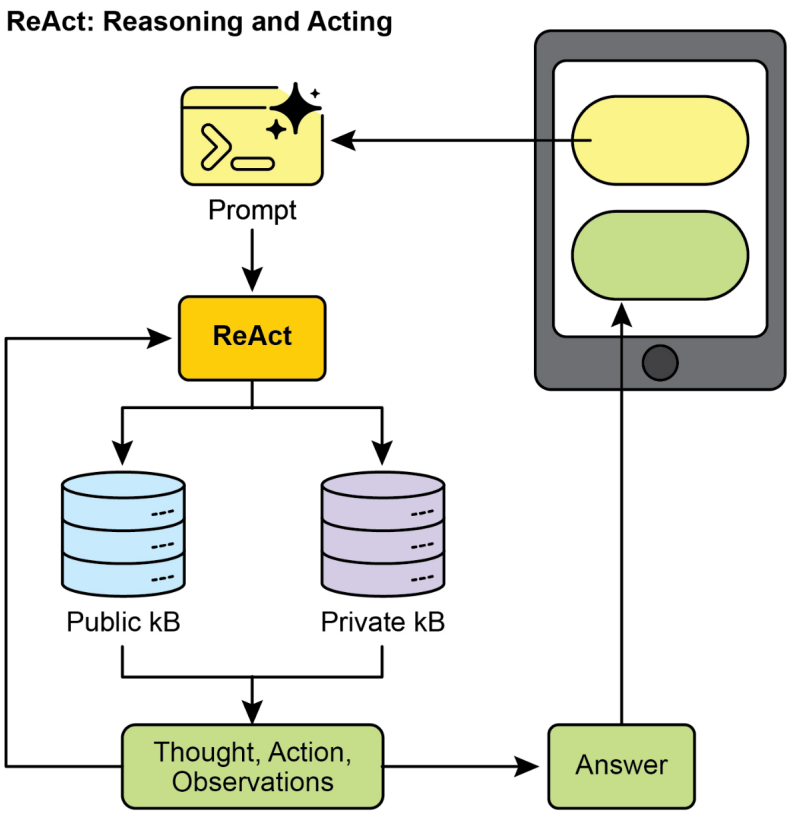


Fig. 1: CoT prompt alongside the detailed, step-by-step response generated by the agent.

图 1：CoT 提示与智能体生成的分步详细回答

Let's see an example. It begins with a set of instructions that tell the AI how to think, defining its persona and a clear five-step process to follow. This is the prompt that initiates structured thinking.

下面举例：先用一套说明规定 AI 如何思考——设定角色并给出清晰的五步流程，也就是启动结构化推理的提示。

Following that, the example shows the CoT process in action. The section labeled "Agent's Thought Process" is the internal monologue where the model executes the instructed steps. This is the literal "chain of thought." Finally, the "Agent's Final Answer" is the polished, comprehensive output generated as a result of that careful, step-by-step reasoning process

随后示例展示 CoT 如何落地：标注为「Agent's Thought Process」的部分，是模型按步骤执行的内心独白，也就是字面上的「思维链」；「Agent's Final Answer」则是经过逐步推理后整理好的最终回答。

```
You are an Information Retrieval Agent. Your goal is to answer the user's question comprehensively and accurately by thinking step-by-step.

Here's the process you must follow:
1.  **Analyze the Query:** Understand the core subject and specific requirements of the user's question. Identify key entities, keywords, and the type of
information being sought.
2.  **Formulate Search Queries (for Knowledge Base):** Based on your analysis, generate a list of precise search queries that you would use to retrieve
relevant information from a knowledge base or external tools.
3.  **Simulate Information Retrieval (Self-Correction/Reasoning):** For each search query, mentally consider what kind of information you expect to find. If
you were to retrieve the content, what would be the most relevant snippets? Think about potential ambiguities or missing pieces.
4.  **Synthesize
Information:** Based on the simulated retrieval and your understanding of the user's original query, synthesize the gathered information into a coherent and
complete answer. Ensure all aspects of the query are addressed.
5.  **Review and Refine:** Before finalizing, critically evaluate your answer. Is it accurate? Is it comprehensive? Is it easy to understand? Is it concise?
If not, identify what needs to be improved and how.

**User Query:** "Explain the main differences between classical computers and quantum computers, and briefly describe a potential application of quantum
computing."

**Agent's Thought Process (Internal CoT Output - encouraged by the prompt):**

**Thought 1 (Analyze Query):**
The user wants two things:
    1.  Main differences between classical and quantum computers.
    2.  One potential application of quantum computing. Keywords: classical computers, quantum computers, differences, potential application.

**Thought 2 (Formulate Search Queries):**
    - "differences classical vs quantum computers"
    - "how quantum computers work"
    - "how classical computers work"
    - "applications of quantum computing"

**Thought 3 (Simulate Information Retrieval & Identify Key Concepts):**
    - From "differences": Expect concepts like bits vs. qubits, superposition, entanglement, probabilistic nature, computational power for specific problems.
    - From "how they work": Classical uses transistors/binary. Quantum uses quantum mechanics.
    - From "applications": Expect drug discovery, material science, cryptography, optimization.

**Thought 4 (Synthesize Information):**
Classical computers use bits (0 or 1), processing information sequentially. Quantum computers use qubits, which can be 0, 1, or both simultaneously
(superposition), and can be entangled, allowing for vastly different computational approaches. This enables them to solve certain problems (like factoring
large numbers or simulating complex molecules) far more efficiently than classical computers. A potential application is in drug discovery, where quantum
computers could simulate molecular interactions with unprecedented accuracy, accelerating the development of new medicines.

**Thought 5 (Review and Refine):**
    - Differences are covered (bits vs qubits, superposition, entanglement, processing).
    - Application is given (drug discovery). - Is it brief? Yes. - Is it clear? Yes. - Looks good.

**Agent's Final Answer (Output to User):**
Classical computers process information using bits, which can be either a 0 or a 1 at any given time, performing operations sequentially. In contrast, quantum
computers utilize qubits, which can exist in multiple states simultaneously through superposition and can be linked through entanglement, allowing for
exponentially more complex computations. This fundamental difference enables quantum machines to tackle certain problems, such as simulating molecular
structures or breaking complex encryption, that are intractable for even the most powerful classical supercomputers. A significant potential application of
quantum computing lies in drug discovery, where its ability to precisely model molecular behavior could revolutionize the development of new pharmaceuticals.
```

Tree-of-Thought (ToT) is a reasoning technique that builds upon Chain-of-Thought (CoT). It allows large language models to explore multiple reasoning paths by branching into different intermediate steps, forming a tree structure (see Fig. 2) This approach supports complex problem-solving by enabling backtracking, self-correction, and exploration of alternative solutions. Maintaining a tree of possibilities allows the model to evaluate various reasoning trajectories before finalizing an answer. This iterative process enhances the model's ability to handle challenging tasks that require strategic planning and decision-making.

思维树（**Tree-of-Thought, ToT**） 在 CoT 之上进一步发展，允许 LLM 在中间步骤处分叉，探索多条推理路径，形成树状结构（见图 2）。借助回溯、自我纠正与对备选方案的搜索，它更胜任复杂问题；在给出终稿前比较多种推理轨迹，迭代过程能强化模型在需要战略规划与决策的任务上的表现。

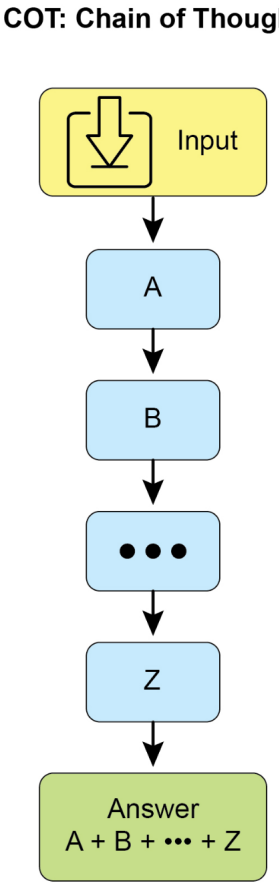


Fig.2: Example of Tree of Thoughts

图 2：思维树示例

Self-correction, also known as self-refinement, is a crucial aspect of an agent's reasoning process, particularly within Chain-of-Thought prompting. It involves the agent's internal evaluation of its generated content and intermediate thought processes. This critical review enables the agent to identify ambiguities, information gaps, or inaccuracies in its understanding or solutions. This iterative cycle of reviewing and refining allows the agent to adjust its approach, improve response quality, and ensure accuracy and thoroughness before delivering a final output. This internal critique enhances the agent's capacity to produce reliable and high-quality results, as demonstrated in examples within the dedicated Chapter 4.

自我纠正（**Self-correction**） 也称自我精炼，是智能体推理中的关键环节，在 CoT 中尤为常见：智能体会审视已生成内容与中间思考，发现歧义、信息缺口或理解、解答中的偏差；通过“审阅—改进”的循环调整策略、提升回答质量，并在交付前尽量保证准确与完整。这种内部审视有助于获得更可靠、更高质量的输出；更多示例见第 4 章。

This example demonstrates a systematic process of self-correction, crucial for refining AI-generated content. It involves an iterative loop of drafting, reviewing against original requirements, and implementing specific improvements. The illustration begins by outlining the AI's function as a "Self-Correction Agent" with a defined five-step analytical and revision workflow. Following this, a subpar "Initial Draft" of a social media post is presented. The "Self-Correction Agent's Thought Process" forms the core of the demonstration. Here, the Agent critically evaluates the draft according to its instructions, pinpointing weaknesses such as low engagement and a vague call to action. It then suggests concrete enhancements, including the use of more impactful verbs and emojis. The process concludes with the "Final Revised Content," a polished and notably improved version that integrates the self-identified adjustments.

本例演示一套可复用的自我纠正流程，对打磨 AI 生成内容尤为有效：起草 → 对照原始要求审阅 → 落实具体修改，循环往复。文内先定义「Self-Correction Agent」与五步分析—修订工作流；再展示一则较弱的社交媒体草稿「Initial Draft」；重点在「Self-Correction Agent's Thought Process」——按指令识别问题，例如互动感不足、行动号召含糊，并建议改用更有力的动词、适当加入表情符号等；最后给出吸收上述修改的「Final Revised Content」。

```
You are a highly critical and detail-oriented Self-Correction Agent. Your task is to review a previously generated piece of content against its original
requirements and identify areas for improvement. Your goal is to refine the content to be more accurate, comprehensive, engaging, and aligned with the
prompt.
```

```
Here's the process you must follow for self-correction:

1.  **Understand Original Requirements:** Review the initial prompt/requirements that led to the content's creation. What was the *original
intent*? What were the key constraints or goals?
2.  **Analyze Current Content:** Read the provided content carefully.
3.  **Identify Discrepancies/Weaknesses:** Compare the current content against the original requirements. Look for:
    * **Accuracy Issues:** Are there any factual errors or misleading statements?
    * **Completeness Gaps:** Does it fully address all aspects of the original prompt? Is anything missing?
    * **Clarity & Coherence:** Is the language clear, concise, and easy to understand? Does it flow logically?
    * **Tone & Style:** Does it match the desired tone and style (e.g., professional, engaging, concise)?
    * **Engagement:** Is it captivating? Does it hold the reader's attention?
    * **Redundancy/Verbosity:** Can any parts be condensed or removed without losing meaning?
4.  **Propose Specific Improvements:** For each identified weakness, suggest concrete and actionable changes. Do not just state the problem; propose a
solution.
5.  **Generate Revised Content:** Based on your proposed improvements, rewrite the original content to incorporate all the necessary changes. Ensure the
revised content is polished and ready for final use.

**Original Prompt/Requirements:** "Write a short, engaging social media post (max 150 characters) announcing a new eco-friendly product line: 'GreenTech
Gadgets'."

**Initial Draft (Content to be corrected):** "We have new products. They are green and techy. Buy GreenTech Gadgets now!"

**Self-Correction Agent's Thought Process (Internal CoT Output - encouraged by the prompt):**

**Thought 1 (Understand Original Requirements):**
    - Short, engaging social media post.
    - Max 150 characters.
    - Announce "GreenTech Gadgets" - a new eco-friendly product line.

**Thought 2 (Analyze Current Content):**
"We have new products. They are green and techy. Buy GreenTech Gadgets now!" (64 characters)

**Thought 3 (Identify Discrepancies/Weaknesses):** - **Engaging:** Not very engaging. Sounds generic. - **Eco-friendly:** Implied by "green," but not
explicitly highlighted as a benefit. - **Call to Action:** "Buy now!" is okay, but could be more compelling. - **Conciseness:** It's short, but could use
stronger verbs/phrasing. - **Impact:** Lacks excitement for a new product launch.

**Thought 4 (Propose Specific
Improvements):** - Make it more dynamic and exciting. - Emphasize "eco-friendly" and "innovation." - Use stronger verbs and more active voice. - Add a clear,
engaging call to action. - Ensure it stays under 150 characters. **Thought 5 (Generate Revised
Content):** Original: "We have new products. They are green and techy. Buy GreenTech Gadgets now!" Revised idea 1: "🌱 Discover GreenTech Gadgets! Our new
eco-friendly line blends innovation with sustainability. Go green, go smart! Shop now! #EcoFriendly #GreenTech" (148 characters) This revision directly
addresses eco-friendliness, uses emojis for engagement, adds a stronger call to action, and includes relevant hashtags while staying within the character
limit. **Self-Correction Agent's Final Revised Content (Output to User):** 🌱 Discover GreenTech Gadgets! Our new eco-friendly line blends innovation with
sustainability. Go green, go smart! Shop now! #EcoFriendly #GreenTech
```

Fundamentally, this technique integrates a quality control measure directly into the Agent's content generation, yielding more refined, precise, and superior results that more effectively meet intricate user demands.

归根结底，这项技术把质量控制嵌进智能体的内容生成链路，产出更精炼、更精确、质量更高的结果，从而更好满足复杂场景下的用户需求。

Program-Aided Language Models (PALMs) integrate LLMs with symbolic reasoning capabilities. This integration allows the LLM to generate and execute code, such as Python, as part of its problem-solving process. PALMs offload complex calculations, logical operations, and data manipulation to a deterministic programming environment. This approach utilizes the strengths of traditional programming for tasks where LLMs might exhibit limitations in accuracy or consistency. When faced with symbolic challenges, the model can produce code, execute it, and convert the results into natural language. This hybrid methodology combines the LLM's understanding and generation abilities with precise computation, enabling the model to address a wider range of complex problems with potentially increased reliability and accuracy. This is important for agents as it allows them to perform more accurate and reliable actions by leveraging precise computation alongside their understanding and generation capabilities. An example is the use of external tools within Google's ADK for generating code.

程序辅助语言模型（**PALMs**） 把 LLM 与符号推理结合起来：解题时由模型生成并执行 Python 等代码，把繁重计算、逻辑与数据操作交给确定性运行时，弥补纯语言模型在数值精度或行为一致性上的短板；遇到符号化任务时，可写代码、执行再把结果译回自然语言。这种混合范式同时发挥语义理解与精确计算，拓宽可解问题类型，并有望提高可靠性与正确率。对智能体来说，意味着在「会写会说」之外还能靠代码做更稳的动作；谷歌 ADK 里用外部工具生成代码就是典型用法之一。

```
from google.adk.tools import agent_tool
from google.adk.agents import Agent
from google.adk.tools import google_search
from google.adk.code_executors import BuiltInCodeExecutor


search_agent = Agent(
    model="gemini-2.0-flash",
    name="SearchAgent",
    instruction="""
    You're a specialist in Google Search
    """,
    tools=[google_search],
)

coding_agent = Agent(
    model="gemini-2.0-flash",
    name="CodeAgent",
    instruction="""
```

```
    You're a specialist in Code Execution
    """,
    code_executor=BuiltInCodeExecutor(),
)

root_agent = Agent(
    name="RootAgent",
    model="gemini-2.0-flash",
    description="Root Agent",
    tools=[
        agent_tool.AgentTool(agent=search_agent),
        agent_tool.AgentTool(agent=coding_agent),
    ],
)
```

Reinforcement Learning with Verifiable Rewards (RLVR): While effective, the standard Chain-of-Thought (CoT) prompting used by many LLMs is a somewhat basic approach to reasoning. It generates a single, predetermined line of thought without adapting to the complexity of the problem. To overcome these limitations, a new class of specialized "reasoning models" has been developed. These models operate differently by dedicating a variable amount of "thinking" time before providing an answer. This "thinking" process produces a more extensive and dynamic Chain-of-Thought that can be thousands of tokens long. This extended reasoning allows for more complex behaviors like self-correction and backtracking, with the model dedicating more effort to harder problems. The key innovation enabling these models is a training strategy called Reinforcement Learning from Verifiable Rewards (RLVR). By training the model on problems with known correct answers (like math or code), it learns through trial and error to generate effective, long-form reasoning. This allows the model to evolve its problem-solving abilities without direct human supervision. Ultimately, these reasoning models don't just produce an answer; they generate a "reasoning trajectory" that demonstrates advanced skills like planning, monitoring, and evaluation. This enhanced ability to reason and strategize is fundamental to the development of autonomous AI agents, which can break down and solve complex tasks with minimal human intervention.

可验证奖励的强化学习（**RLVR**）：许多 LLM 沿用的标准 CoT 虽有用，但偏朴素：往往只走一条既定思路，很少随题目难度调节。为突破这一局限，业界出现了专门的「推理模型」：在给出答案前分配可变的「思考」预算，生成长得多、也灵活得多的思维链（可达数千 token），从而支持自我纠正、回溯等更复杂行为，并对更难的问题自动多花时间。背后的关键训练范式仍是 RLVR：在答案可核对的问题（如数学、代码）上，用试错学习把长程推理练出来，使模型在人类不逐条监督的情况下也能进化解题策略。这类模型不仅返回答案，还会留下体现规划、监控与评估等能力的「推理轨迹」；更强的推理与策略水平，是打造「少人工介入即可拆解并完成复杂任务」的自主 智能体的基础。

ReAct (Reasoning and Acting, see Fig. 3, where KB stands for Knowledge Base) is a paradigm that integrates Chain-of-Thought (CoT) prompting with an agent's ability to interact with external environments through tools. Unlike generative models that produce a final answer, a ReAct agent reasons about which actions to take. This reasoning phase involves an internal planning process, similar to CoT, where the agent determines its next steps, considers available tools, and anticipates outcomes. Following this, the agent acts by executing a tool or function call, such as querying a database, performing a calculation, or interacting with an API.

ReAct（Reasoning and Acting，见图 3；KB 指知识库）将 CoT 与“通过工具同外部环境交互”的能力结合起来。不同于只负责生成最终答案的生成式模型，ReAct 智能体需要先明确下一步应采取的行动：推理阶段类似 CoT 的内部规划，用于选定下一步、盘点可用工具并预判后果；随后再实际执行工具或函数调用（如查询知识库、进行计算、调用 API 等）。

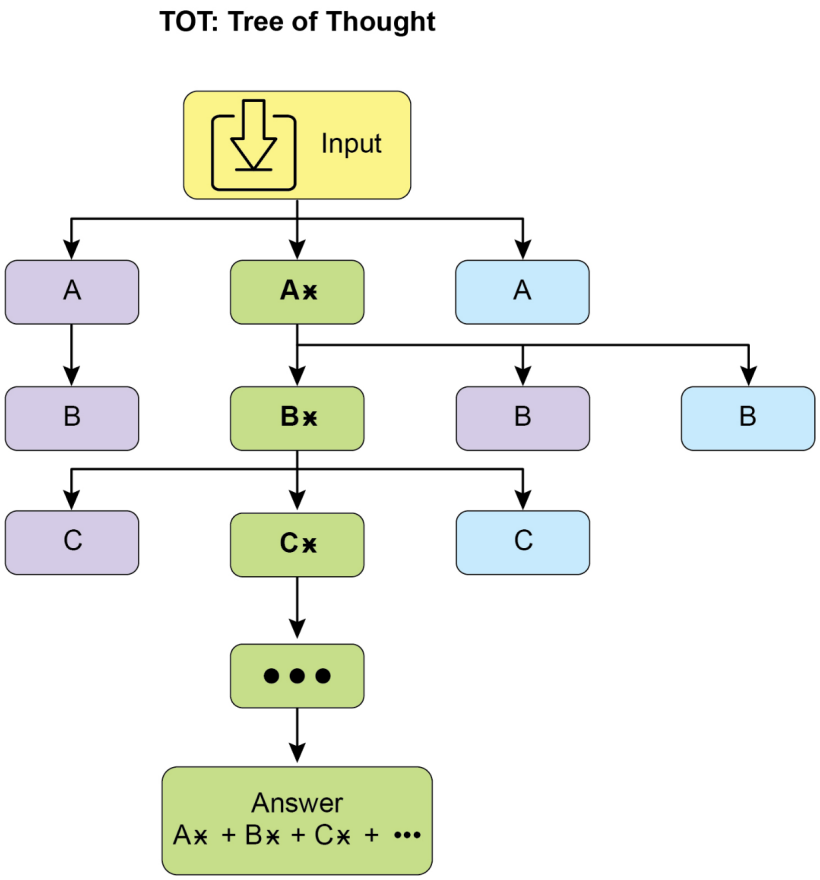


Fig.3: Reasoning and Act

图 3：推理与行动（ReAct）

ReAct operates in an interleaved manner: the agent executes an action, observes the outcome, and incorporates this observation into subsequent reasoning. This iterative loop of "Thought, Action, Observation, Thought..." allows the agent to dynamically adapt its plan, correct errors, and achieve goals requiring multiple interactions with the environment. This provides a more robust and flexible problem-solving approach compared to linear CoT, as the agent responds to real-time feedback. By combining language model understanding and generation with the capability to use tools, ReAct enables agents to perform complex tasks requiring both reasoning and practical execution. This approach is crucial for agents as it allows them to not only reason but also to practically execute steps and interact with dynamic environments.

ReAct 以交错方式运行：先行动，再接收环境反馈，再将观察结果写回下一轮推理。「思考 → 行动 → 观察 → 再思考……」的闭环使智能体能够调整计划、纠正偏差，并完成需要与环境多轮交互的目标。相比线性的 CoT，它更能适应真实世界中的变化，因为每一步都能获得实时反馈。通过将语言模型的理解与生成能力同工具调用结合起来，ReAct 适用于既需要推理、又需要执行的复杂任务；其对智能体范式的重要性在于，它不仅停留于推理层面，还能在动态环境中逐步完成实际任务。

CoD (Chain of Debates) is a formal AI framework proposed by Microsoft where multiple, diverse models collaborate and argue to solve a problem, moving beyond a single AI's "chain of thought." This system operates like an AI council meeting, where different models present initial ideas, critique each other's reasoning, and exchange counterarguments. The primary goal is to enhance accuracy, reduce bias, and improve the overall quality of the final answer by leveraging collective intelligence. Functioning as an AI version of peer review, this method creates a transparent and trustworthy record of the reasoning process. Ultimately, it represents a shift from a solitary Agent providing an answer to a collaborative team of Agents working together to find a more robust and validated solution.

CoD（Chain of Debates，辩论链） 是微软提出的一种正式框架：让多个不同模型通过协作辩论来共同求解问题，从而突破单模型单条"思维链"的局限。其运作方式类似 AI 理事会：各方先提出观点，再对彼此的推理链条进行质询与反驳；借助集体智能提升准确率、降低偏见，并改善最终答案的质量。它近似于 AI 版的同行评审，能够保留可追溯、可核验的推理过程，也标志着系统正从"一个智能体给出答案"走向"多个智能体共同打磨更稳健、更可验证的结论"。

GoD (Graph of Debates) is an advanced Agentic framework that reimagines discussion as a dynamic, non-linear network rather than a simple chain. In this model, arguments are individual nodes connected by edges that signify relationships like 'supports' or 'refutes,' reflecting the multi-threaded nature of real debate. This structure allows new lines of inquiry to dynamically branch off, evolve independently, and even merge over time. A conclusion is reached not at the end of a sequence, but by identifying the most robust and well-supported cluster of arguments within the entire graph. In this context, "well-supported" refers to knowledge that is firmly established and verifiable. This can include information considered to be ground truth, which means it is inherently correct and widely accepted as fact. Additionally, it encompasses factual evidence obtained through search grounding, where information is validated against external sources and real-world data. Finally, it also pertains to a consensus reached by multiple models during a debate, indicating a high degree of agreement and confidence in the information presented. This comprehensive approach ensures a more robust and reliable foundation for the information being discussed. This approach provides a more holistic and realistic model for complex, collaborative AI reasoning.

GoD（Graph of Debates，辩论图） 是更进一层的智能体框架：把讨论画成一张动态、非线性的图，而不是一条直线。每个论点是一个节点，边表示「支持」「反驳」等关系，更贴近真实辩论里多线并进的样子；新的追问可以随时分叉、各自演进，甚至以后再汇合。最终结论不必出现在某条链的末尾，而是从整张图里挑出证据最充分、支撑最强的那一簇论点。「支撑充分」意味着知识站得住脚且可核对：可以是广泛接受的 ground truth、经搜索 grounding 与外部来源交叉验证的事实，也可以是多模型辩论后形成的高共识、高置信信息。这样为集体推理提供了更扎实的事实底座，也比单链叙事更贴近复杂的真实讨论。

MASS (optional advanced topic): An in-depth analysis of the design of multi-agent systems reveals that their effectiveness is critically dependent on both the quality of the prompts used to program individual agents and the topology that dictates their interactions. The complexity of designing these systems is significant, as it involves a vast and intricate search space. To address this challenge, a novel framework called Multi-Agent System Search (MASS) was developed to automate and optimize the design of MAS.

MASS（可选进阶）： 细究多智能体系统（MAS）的设计会发现，成败很大程度上取决于：给每个智能体写的提示是否到位，以及它们之间如何连接、如何传递任务（拓扑）。设计空间巨大且彼此纠缠，手工试错很难穷尽。为此研究者提出多智能体系统搜索（MASS），用自动化搜索同时优化提示与拓扑。

MASS employs a multi-stage optimization strategy that systematically navigates the complex design space by interleaving prompt and topology optimization (see Fig. 4)

MASS 采用多阶段策略，在庞大设计空间里交替优化「提示怎么写」和「智能体怎么连」（见图 4），系统性地向前搜索。

1. Block-Level Prompt Optimization: The process begins with a local optimization of prompts for individual agent types, or "blocks," to ensure each component performs its role effectively before being integrated into a larger system. This initial step is crucial as it ensures that the subsequent topology optimization builds upon well-performing agents, rather than suffering from the compounding impact of poorly configured ones. For example, when optimizing for the HotpotQA dataset, the prompt for a "Debator" agent is creatively framed to instruct it to act as an "expert fact-checker for a major publication". Its optimized task is to meticulously review proposed answers from other agents, cross-reference them with provided context passages, and identify any inconsistencies or unsupported claims. This specialized role-playing prompt, discovered during block-level optimization, aims to make the debator agent highly effective at synthesizing information before it's even placed into a larger workflow

1. 块级提示优化： 先按“智能体类型 / 功能块”分别优化提示，使每个模块在接入大系统之前就具备独立完成职责的能力；这一步极为关键，否则后续调整拓扑只会放大局部配置不佳的问题。以 HotpotQA 为例，“Debator”智能体的提示可被优化为“主流媒体的事实核查专家”：负责逐条核对其他智能体给出的答案，对照原文段落，识别不一致或缺乏依据的表述。块级搜索得到的这类角色化提示，能够让“辩论 / 挑错”模块在尚未嵌入完整流水线之前，就具备较强的信息整合能力。

2. Workflow Topology Optimization: Following local optimization, MASS optimizes the workflow topology by selecting and arranging different agent interactions from a customizable design space. To make this search efficient, MASS employs an influence-weighted method. This method calculates the "incremental influence" of each topology by measuring its performance gain relative to a baseline agent and uses these scores to guide the search toward more promising combinations. For instance, when optimizing for the MBPP coding task, the topology search discovers that a specific hybrid workflow is most effective. The best-found topology is not a simple structure but a combination of an iterative refinement process with external tool use. Specifically, it consists of one predictor agent that engages in several rounds of reflection, with its code being verified by one executor agent that runs the code against test cases. This discovered workflow shows that for coding, a structure that combines iterative self-correction with external verification is superior to simpler MAS designs

2. 工作流拓扑优化： 块级提示稳定之后，MASS 会在可配置的设计空间里挑选、排列智能体之间的连边与调用顺序，专门优化工作流拓扑。为控制搜索成本，使用「影响加权」思路：相对某个基线智能体，估计每种拓扑带来的边际收益，用分数将搜索引向更有前景的组合。以 MBPP 编程题为例，搜索到的最优形态往往不是「一条直线」，而是「多轮自我修订 + 外部工具」的混合体：一个负责编写代码并进行反思的预测智能体，配合一个在测试用例上实际执行代码的执行智能体。这说明对编码场景，将迭代式自我纠正与外部执行验证结合起来，往往优于结构更简单的 MAS。

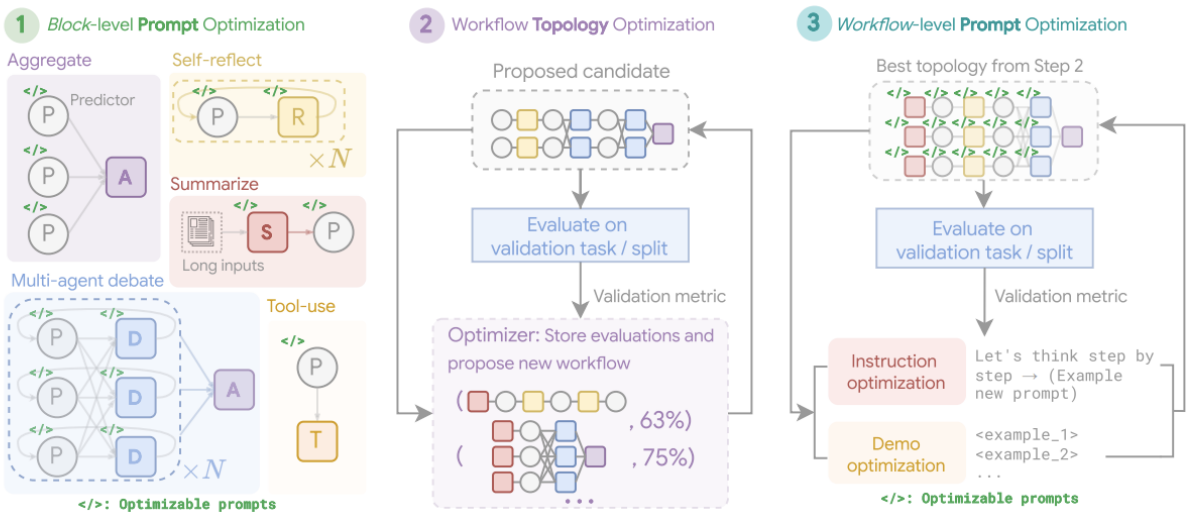


Fig. 4: (Courtesy of the Authors): The Multi-Agent System Search (MASS) Framework is a three-stage optimization process that navigates a search space encompassing optimizable prompts (instructions and demonstrations) and configurable agent building blocks (Aggregate, Reflect, Debate, Summarize, and Tool-use). The first stage, Block-level Prompt Optimization, independently optimizes prompts for each agent module. Stage two, Workflow Topology Optimization, samples valid system configurations from an influence-weighted design space, integrating the optimized prompts. The final stage, Workflow-level Prompt Optimization, involves a second round of prompt optimization for the entire multi-agent system after the optimal workflow from Stage two has been identified

图 4（作者供图）：多智能体系统搜索（MASS）分三阶段推进，搜索对象既包括可调的提示（系统指令与 few-shot 示例），也包括可拼装的智能体积木（聚合、反思、辩论、摘要、工具调用等）。阶段一在模块粒度上单独优化提示；阶段二在带影响权重的设计空间里采样合法拓扑，并把阶段一的提示嵌进去；阶段三在已锁定的最佳拓扑上，对整个系统再做一轮全局提示微调。

3. Workflow-Level Prompt Optimization: The final stage involves a global optimization of the entire system's prompts. After identifying the best-performing topology, the prompts are fine-tuned as a single, integrated entity to ensure they are tailored for orchestration and that agent interdependencies are optimized. As an example, after finding the best topology for the DROP dataset, the final optimization stage refines the "Predictor" agent's prompt. The final, optimized prompt is highly detailed, beginning by providing the agent with a summary of the dataset itself, noting its focus on "extractive question answering" and "numerical information". It then includes few-shot examples of correct question-answering behavior and frames the core instruction as a high-stakes scenario: "You are a highly specialized AI tasked with extracting critical numerical information for an urgent news report. A live broadcast is relying on your accuracy and speed". This multi-faceted prompt, combining meta-knowledge, examples, and role-playing, is tuned specifically for the final workflow to maximize accuracy

3. 工作流级提示优化： 最后一轮把整套提示当作一个整体来调：拓扑已定，接下来要让各智能体的措辞彼此咬合、减少隐性冲突。以 DROP 为例，最终优化会重点打磨「Predictor」的提示——先交代数据集特点（抽取式问答、偏重数值），再塞入高质量 few-shot，最后用强情境锚定职责，例如：「你是为突发新闻直播抽关键数字的专员，准确和速度同样要命」。把元知识、示范与角色设定叠在一起，针对整条流水线做联合微调，以把端到端准确率推上去。

Key Findings and Principles: Experiments demonstrate that MAS optimized by MASS significantly outperform existing manually designed systems and other automated design methods across a range of tasks. The key design principles for effective MAS, as derived from this research, are threefold

主要结论与设计原则：实验显示，经 MASS 搜出的 MAS 在多项基准上明显强于人工精心设计的系统，也优于其他自动化设计管线。论文归纳的三条经验是：

- Optimize individual agents with high-quality prompts before composing them.
- Construct MAS by composing influential topologies rather than exploring an unconstrained search space.
- Model and optimize the interdependencies between agents through a final, workflow-level joint optimization.

- 先用高质量提示充分优化各个智能体，再考虑系统级组合。
- 优先围绕“高影响力拓扑”开展搜索，而不是在完全无结构的巨大空间中盲目探索。
- 拓扑敲定后，再做一轮工作流级联合提示优化，显式处理智能体之间的隐性依赖。

Building on our discussion of key reasoning techniques, let's first examine a core performance principle: the Scaling Inference Law for LLMs. This law states that a model's performance predictably improves as the computational resources allocated to it increase. We can see this principle in action in complex systems like Deep

Research, where an AI agent leverages these resources to autonomously investigate a topic by breaking it down into sub-questions, using Web search as a tool, and synthesizing its findings.

在梳理完推理技术之后，先记住一条与性能直接相关的规律：LLM 的推理扩展定律指出，在推理阶段投入更多计算资源（如更多步骤、更多样本、更复杂的解码策略等），性能通常会呈现可预期的提升。Deep Research 类产品就是例子：智能体将大课题拆分为子问题，反复进行网络检索、交叉比对与综合分析，本质上都是在投入更充裕的推理预算。

Deep Research. The term "Deep Research" describes a category of AI Agentic tools designed to act as tireless, methodical research assistants. Major platforms in this space include Perplexity AI, Google's Gemini research capabilities, and OpenAI's advanced functions within ChatGPT (see Fig.5).

深度研究（**Deep Research**） 泛指一类「像不知疲倦、又有章法的助理」的 智能体工具，替你做多轮检索与写作。代表性产品包括 Perplexity AI、Google Gemini 的深度研究能力，以及 ChatGPT 中的高阶研究模式等（见图 5）。

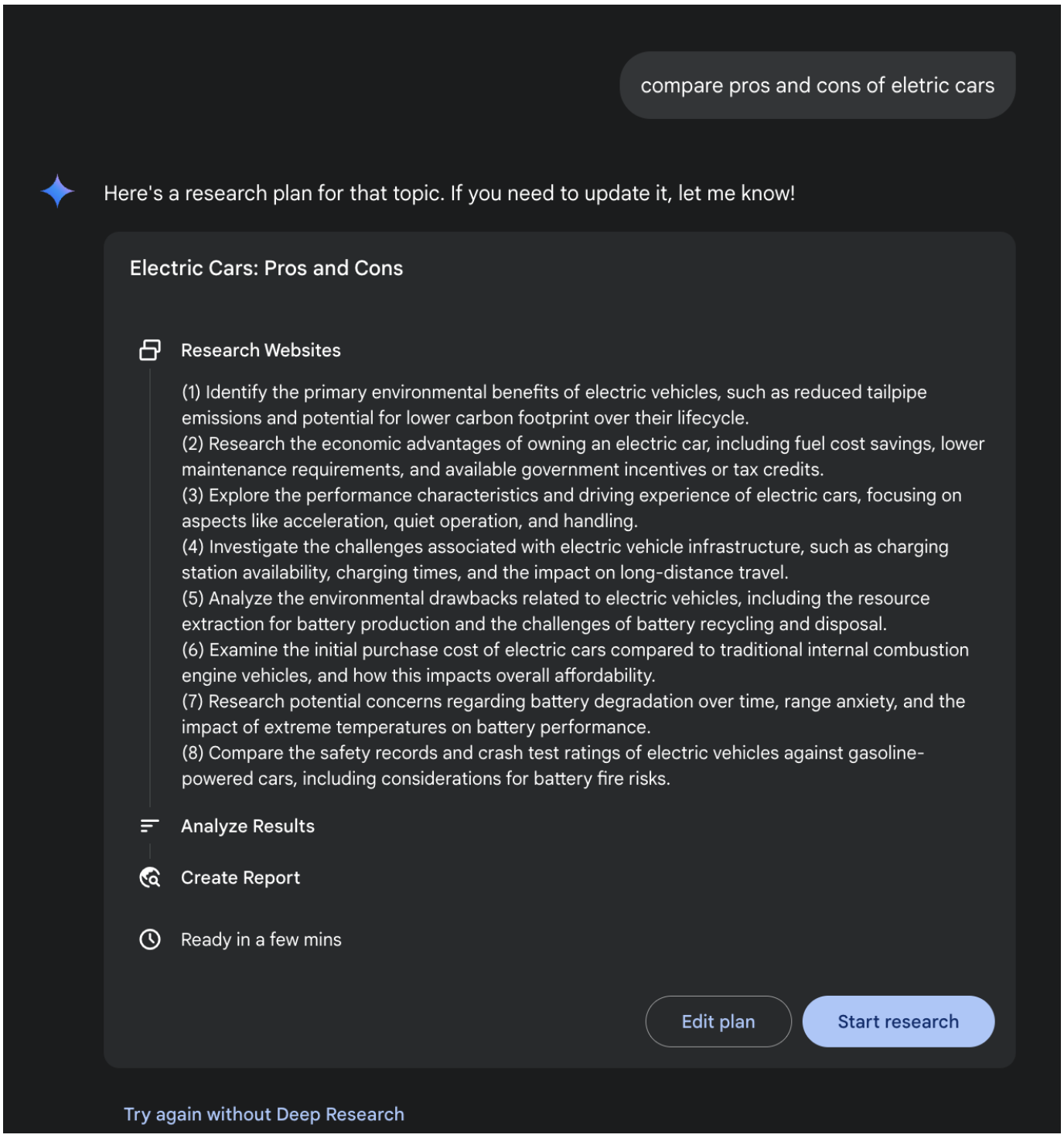


Fig. 5: Google Deep Research for Information Gathering

图 5：谷歌深度研究用于信息收集

A fundamental shift introduced by these tools is the change in the search process itself. A standard search provides immediate links, leaving the work of synthesis to you. Deep Research operates on a different model. Here, you task an AI with a complex query and grant it a "time budget"—usually a few minutes. In return for this patience, you receive a detailed report.

它们真正改变的是「搜索」这件事的形态：传统搜索秒回一堆链接，综合与取舍仍靠你自己；深度研究则反过来——你抛出复杂问题，并愿意等几分钟的「时间预算」，换一份已经帮你读完、比对、写好的长报告。

During this time, the AI works on your behalf in an agentic way. It autonomously performs a series of sophisticated steps that would be incredibly time-consuming for a person:

等待的这段时间里，AI 以智能体方式替你干活，自主完成一整套对人来说极其费时的流程：

1. Initial Exploration: It runs multiple, targeted searches based on your initial prompt.
2. Reasoning and Refinement: It reads and analyzes the first wave of results, synthesizes the findings, and critically identifies gaps, contradictions, or areas that require more detail.
3. Follow-up Inquiry: Based on its internal reasoning, it conducts new, more nuanced searches to fill those gaps and deepen its understanding.
4. Final Synthesis: After several rounds of this iterative searching and reasoning, it compiles all the validated information into a single, cohesive, and structured summary.

1. 初始探索：围绕你的问题发起多轮、各有侧重的检索。
2. 推理与精炼：阅读首轮结果，做归纳，并主动标出缺口、矛盾或仍需深挖之处。
3. 跟进查询：根据内部判断再开更细的新检索，补齐证据链。
4. 终稿综合：多轮搜索—思考之后，把核对过的材料压成一份结构清楚、前后连贯的长文摘要。

This systematic approach ensures a comprehensive and well-reasoned response, significantly enhancing the efficiency and depth of information gathering, thereby facilitating more agentic decision-making.

这种流水线式方法更容易给出覆盖面广、论证相对充分的回答，把信息收集的效率和深度一起抬上去，也更贴近「智能体化」的决策辅助。

Scaling Inference Law

推理扩展定律

This critical principle dictates the relationship between an LLM's performance and the computational resources allocated during its operational phase, known as inference. The Inference Scaling Law differs from the more familiar scaling laws for training, which focus on how model quality improves with increased data volume and computational power during a model's creation. Instead, this law specifically examines the dynamic trade-offs that occur when an LLM is actively generating an output or answer.

这条规律描述的是：在推理阶段多投入计算，会如何改变 LLM 的表现。它和我们更常听说的「训练扩展定律」不是一回事——训练定律讲的是数据、参数、训练算力与模型质量的关系；推理定律则聚焦模型已经部署好之后，在真正为用户生成答案时，算力与时间预算之间的取舍。

A cornerstone of this law is the revelation that superior results can frequently be achieved from a comparatively smaller LLM by augmenting the computational investment at inference time. This doesn't necessarily mean using a more powerful GPU, but rather employing more sophisticated or resource-intensive inference strategies. A prime example of such a strategy is instructing the model to generate multiple potential answers—perhaps through techniques like diverse beam search or self-consistency methods—and then employing a selection mechanism to identify the most optimal output. This iterative refinement or multiple-candidate generation process demands more computational cycles but can significantly elevate the quality of the final response.

其中一条关键洞见是：不一定换更大的模型，只要在推理时舍得用算力，较小的 LLM 有时也能反超。「多给算力」未必等于换更贵的 GPU，更多是指更费时的解码策略——例如并行生成多份答案（多样化束搜索、自洽投票等），再用规则或裁判模型挑出最优。多跑几轮、多采样几次，会吃掉更多 token 与延迟，但往往能把最终回答的质量明显拉上去。

This principle offers a crucial framework for informed and economically sound decision-making in the deployment of Agents systems. It challenges the intuitive notion that a larger model will always yield better performance. The law posits that a smaller model, when granted a more substantial "thinking budget" during inference, can occasionally surpass the performance of a much larger model that relies on a simpler, less computationally intensive generation process. The "thinking

budget" here refers to the additional computational steps or complex algorithms applied during inference, allowing the smaller model to explore a wider range of possibilities or apply more rigorous internal checks before settling on an answer.

这为部署智能体时的成本—效果权衡提供了清晰框架，也打破了“参数越大就一定越强”的简单直觉：小模型如果在推理端获得更充裕的“思考预算”，有时可能超过仅依赖单次生成的大模型。这里的“思考预算”泛指额外的推理步数、更复杂的搜索或验证流程，使小模型在作出最终判断前能够探索更多路径并进行更多自检。

Consequently, the Scaling Inference Law becomes fundamental to constructing efficient and cost-effective Agentic systems. It provides a methodology for meticulously balancing several interconnected factors:

因此，推理扩展定律是搭建「又省又好用」的智能体系统时的底层逻辑之一，迫使你在以下几件事之间做精细取舍：

- **Model Size:** Smaller models are inherently less demanding in terms of memory and storage.
- **Response Latency:** While increased inference-time computation can add to latency, the law helps identify the point at which the performance gains outweigh this increase, or how to strategically apply computation to avoid excessive delays.
- **Operational Cost:** Deploying and running larger models typically incurs higher ongoing operational costs due to increased power consumption and infrastructure requirements. The law demonstrates how to optimize performance without unnecessarily escalating these costs.

- 模型规模： 小模型在显存、权重体积和边缘部署上都更友好。
- 响应延迟： 推理侧算力加得越多，端到端时延往往越长；需要判断「质量提升」是否值得这笔延迟，以及能否只在关键路径上加重计算。
- 运营成本： 大模型在线服务的电费、GPU 小时单价、扩容复杂度都更高；推理定律提示你有时可以用「小模型 + 重推理」换到相近效果，从而压低账单。

By understanding and applying the Scaling Inference Law, developers and organizations can make strategic choices that lead to optimal performance for specific agentic applications, ensuring that computational resources are allocated where they will have the most significant impact on the quality and utility of the LLM's output. This allows for more nuanced and economically viable approaches to AI deployment, moving beyond a simple "bigger is better" paradigm.

充分理解这条定律之后，团队便可根据业务场景决定应将算力投入训练、模型规模，还是推理阶段的多步思考流程，把有限预算用于最能提升输出质量与实用性的环节，而不是单纯追求更大的参数规模。

Hands-On Code Example

动手代码示例

The DeepSearch code, open-sourced by Google, is available through the gemini-fullstack-langgraph-quickstart repository (Fig. 6). This repository provides a template for developers to construct full-stack AI agents using Gemini 2.5 and the LangGraph orchestration framework. This open-source stack facilitates experimentation with agent-based architectures and can be integrated with local LLLMs such as Gemma. It utilizes Docker and modular project scaffolding for rapid prototyping. It should be noted that this release serves as a well-structured demonstration and is not intended as a production-ready backend.

Google 开源的 DeepSearch 示例位于 `gemini-fullstack-langgraph-quickstart` 仓库（图 6）。它提供一套模板，演示如何基于 Gemini 2.5 与 LangGraph 搭一个「前端 + 智能体后端」的全栈应用；便于快速试验智能体架构，也可对接 Gemma 等本地模型。项目用 Docker 与分层目录脚手架降低上手成本，但定位仍是结构清晰的参考实现，不要直接当生产后端照搬。

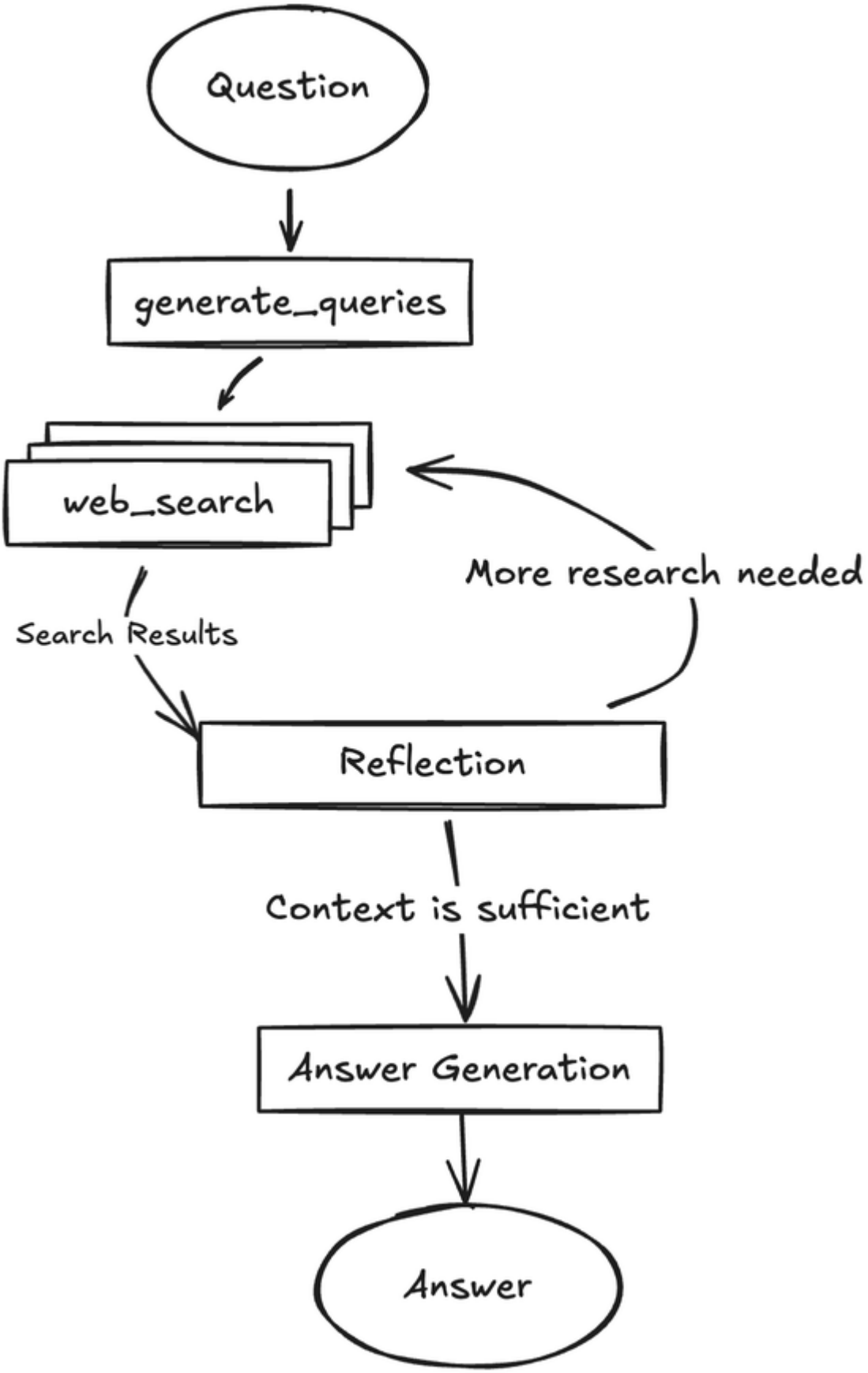

Question
generate_queries
web_search
More research needed
Search Results
Reflection
Context is sufficient
Answer Generation
Answer

Fig. 6: (Courtesy of authors) Example of DeepSearch with multiple Reflection steps

图 6（作者供图）：展示含多轮反思节点的 DeepSearch 示例

This project provides a full-stack application featuring a React frontend and a LangGraph backend, designed for advanced research and conversational AI. A LangGraph agent dynamically generates search queries using Google Gemini models and integrates web research via the Google Search API. The system employs reflective reasoning to identify knowledge gaps, refine searches iteratively, and synthesize answers with citations. The frontend and backend support hot-reloading. The project's structure includes separate frontend/ and backend/ directories. Requirements for setup include Node.js, npm, Python 3.8+, and a Google Gemini API key. After configuring the API key in the backend's .env file, dependencies for both the backend (using pip install .) and frontend (npm install) can be installed. Development servers can be run concurrently with make dev or individually. The backend agent, defined in backend/src/agent/graph.py, generates initial search queries, conducts web research, performs knowledge gap analysis, refines queries iteratively, and synthesizes a cited answer using a Gemini model. Production deployment involves the backend server delivering a static frontend build and requires Redis for streaming real-time output and a Postgres database for managing data. A Docker image can be built and run using docker-compose up, which also requires a LangSmith API key for the docker-compose.yml example. The application utilizes React with Vite, Tailwind CSS, Shadcn UI, LangGraph, and Google Gemini. The project is licensed under the Apache License 2.0.

仓库里是一个 React + LangGraph 的全栈样例，偏「高阶检索 + 对话式研究」场景。图里的智能体会调用 Gemini 动态改写检索 query，经 Google Search API 拉网页；再用反思节点找知识空洞，多轮补搜，最后拼出带来源引用的答案。前后端都支持热更新，代码分 `frontend/` 与 `backend/`。本地开发需要 Node.js、npm、Python 3.8+ 以及 Gemini API Key（写在 backend 的 `.env`）；依赖分别用 `pip install .` 与 `npm install` 安装，可用 `make dev` 一键起双端。智能体拓扑在 `backend/src/agent/graph.py`：从生成查询、联网检索、反思评估，到必要时回到检索或进入终稿节点。若要模拟生产形态，后端会托管打包后的静态前端，并依赖 Redis（流式输出）与 Postgres（状态/数据）；`docker-compose up` 可起全套，示例 `docker-compose.yml` 里还预留了 LangSmith Key。技术栈包括 React、Vite、Tailwind、Shadcn UI、LangGraph、Gemini；许可证为 Apache 2.0。

```
# Create our Agent
Graph builder = StateGraph(OverallState, config_schema=Configuration)
# Define the nodes we will cycle between
builder.add_node("generate_query", generate_query)
builder.add_node("web_research", web_research)
builder.add_node("reflection", reflection)
builder.add_node("finalize_answer", finalize_answer)
# Set the entrypoint as `generate_query`
# This means that this node is the first one called
builder.add_edge(START, "generate_query")
# Add conditional edge to continue with search queries in a parallel branch
builder.add_conditional_edges(
    "generate_query",
    continue_to_web_research,
    ["web_research"]
)
# Reflect on the web research
builder.add_edge("web_research", "reflection")
# Evaluate the research
builder.add_conditional_edges(
    "reflection",
    evaluate_research,
    ["web_research", "finalize_answer"]
)
# Finalize the answer
builder.add_edge("finalize_answer", END)
graph = builder.compile(name="pro-search-agent")
```

Fig.4: Example of DeepSearch with LangGraph (code from backend/src/agent/graph.py)

图 4：DeepSearch + LangGraph 图结构示意（片段出自 `backend/src/agent/graph.py`）

So, what do agents think?

那么，智能体如何「思考」？

In summary, an agent's thinking process is a structured approach that combines reasoning and acting to solve problems. This method allows an agent to explicitly plan its steps, monitor its progress, and interact with external tools to gather information.

概括来说，智能体的「思考」是一套把推理和行动绑在一起的结构化套路：先把步骤想清楚，边做边看进度，需要时调用外部工具补信息。

At its core, the agent's "thinking" is facilitated by a powerful LLM. This LLM generates a series of thoughts that guide the agent's subsequent actions. The process typically follows a thought-action-observation loop:

底层通常是一款能力足够强的 LLM：它持续生成内部推理内容，以指导下一步动作。其经典节奏是思考 → 行动 → 观察，循环往复：

1. **Thought:** The agent first generates a textual thought that breaks down the problem, formulates a plan, or analyzes the current situation. This internal monologue makes the agent's reasoning process transparent and steerable.
2. **Action:** Based on the thought, the agent selects an action from a predefined, discrete set of options. For example, in a question-answering scenario, the action space might include searching online, retrieving information from a specific webpage, or providing a final answer.
3. **Observation:** The agent then receives feedback from its environment based on the action taken. This could be the results of a web search or the content of a webpage.

1. 思考： 先把当前局面用文字捋一遍——拆题、列计划或评估风险；这段独白既是给人类看的审计日志，也方便你用提示继续纠偏。
2. 行动： 在有限的离散动作集合里选一个执行，例如搜索、打开某 URL、调用内部工具，或直接输出最终答案。
3. 观察： 环境把结果喂回来：可能是 SERP 摘要、页面正文，也可能是工具返回的结构化 JSON。

This cycle repeats, with each observation informing the next thought, until the agent determines that it has reached a final solution and performs a "finish" action.

如此循环，观察结果会改写下一轮思考，直到模型自我判断「可以收束」，并触发终止动作。

The effectiveness of this approach relies on the advanced reasoning and planning capabilities of the underlying LLM. To guide the agent, the ReAct framework often employs few-shot learning, where the LLM is provided with examples of human-like problem-solving trajectories. These examples demonstrate how to effectively combine thoughts and actions to solve similar tasks.

效果高度依赖底座模型的规划与常识推理能力。ReAct 论文里通常配合 few-shot，将“人是如何边思考边行动”的轨迹示例提供给模型，教会它在相似任务上复用同一节奏。

The frequency of an agent's thoughts can be adjusted depending on the task. For knowledge-intensive reasoning tasks like fact-checking, thoughts are typically interleaved with every action to ensure a logical flow of information gathering and reasoning. In contrast, for decision-making tasks that require many actions, such as navigating a simulated environment, thoughts may be used more sparingly, allowing the agent to decide when thinking is necessary

「每一步都要不要写思考」也可以按任务调：事实核查这类知识密集型工作，往往每次工具调用前后都写一段推理，保证证据链不断；而在需要连续执行上百步的模拟环境里，则可以仅在关键分叉处显式推理，以将 token 预算更多留给行动。

At a Glance

速览

What: Complex problem-solving often requires more than a single, direct answer, posing a significant challenge for AI. The core problem is enabling AI agents to tackle multi-step tasks that demand logical inference, decomposition, and strategic planning. Without a structured approach, agents may fail to handle intricacies, leading to inaccurate or incomplete conclusions. These advanced reasoning methodologies aim to make an agent's internal "thought" process explicit, allowing it to systematically work through challenges.

是什么： 许多复杂问题无法通过“一句话”完成解答，这正是 智能体面临的核心挑战：它们往往需要逻辑推断、任务分解与战略规划等多步流程。若缺少结构化推理，智能体很容易遗漏细节，进而给出片面甚至错误的结论。上述高级方法的核心，就在于将内部“思考”过程显式化，使系统能够按步骤、有条理地处理复杂问题。

Why: The standardized solution is a suite of reasoning techniques that provide a structured framework for an agent's problem-solving process. Methodologies like Chain-of-Thought (CoT) and Tree-of-Thought (ToT) guide LLMs to break down problems and explore multiple solution paths. Self-Correction allows for the iterative refinement of answers, ensuring higher accuracy. Agentic frameworks like ReAct integrate reasoning with action, enabling agents to interact with external tools and environments to gather information and adapt their plans. This combination of explicit reasoning, exploration, refinement, and tool use creates more robust, transparent, and capable AI systems.

为什么： 业界逐渐形成的一套“标准方法”，就是通过多种推理技术为智能体搭建结构化支架：CoT 与 ToT 帮助模型拆解问题、探索分支；自我纠正负责迭代打磨答案；ReAct 则将推理与工具调用紧密结合，使智能体能够边检索边调整计划。将显式推理、搜索式探索、持续修正与工具使用结合起来，系统会更稳健、更具可解释性，也更能胜任真实任务。

Rule of Thumb: Use these reasoning techniques when a problem is too complex for a single-pass answer and requires decomposition, multi-step logic, interaction with external data sources or tools, or strategic planning and adaptation. They are ideal for tasks where showing the "work" or thought process is as important as the final answer.

经验法则： 只要任务需要拆步、走多跳逻辑、频繁查外部数据或调工具，或者需要根据反馈反复改方案，就该考虑上这些推理技术。若业务方要求「不仅要知道答案，还要能复盘你是怎么想的」，它们几乎必不可少。

Visual Summary:

图示摘要：

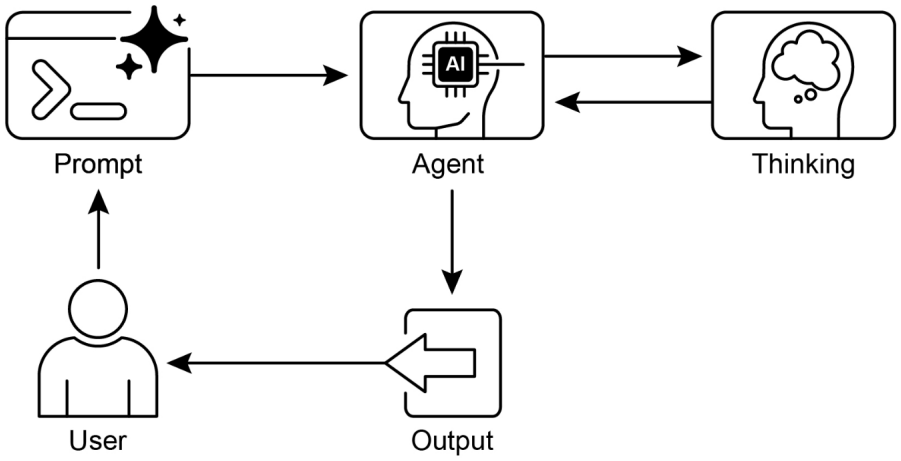


Fig. 7: Reasoning design pattern

图 7：推理设计模式

Key Takeaways

要点

- By making their reasoning explicit, agents can formulate transparent, multi-step plans, which is the foundational capability for autonomous action and user trust.
- The ReAct framework provides agents with their core operational loop, empowering them to move beyond mere reasoning and interact with external tools to dynamically act and adapt within an environment.
- The Scaling Inference Law implies an agent's performance is not just about its underlying model size, but its allocated "thinking time," allowing for more deliberate and higher-quality autonomous actions.
- Chain-of-Thought (CoT) serves as an agent's internal monologue, providing a structured way to formulate a plan by breaking a complex goal into a sequence of manageable actions.
- Tree-of-Thought and Self-Correction give agents the crucial ability to deliberate, allowing them to evaluate multiple strategies, backtrack from errors, and improve their own plans before execution.
- Collaborative frameworks like Chain of Debates (CoD) signal the shift from solitary agents to multi-agent systems, where teams of agents can reason together to tackle more complex problems and reduce individual biases.
- Applications like Deep Research demonstrate how these techniques culminate in agents that can execute complex, long-running tasks, such as in-depth investigation, completely autonomously on a user's behalf.
- To build effective teams of agents, frameworks like MASS automate the optimization of how individual agents are instructed and how they interact, ensuring the entire multi-agent system performs optimally.
- By integrating these reasoning techniques, we build agents that are not just automated but truly autonomous, capable of being trusted to plan, act, and solve complex problems without direct supervision.

- 把推理写出来，智能体才能形成可审计的多步计划——这是自主决策与用户信任的底座。
- ReAct 给出「想—做—看」主循环，让智能体走出纯文本推理，真正与环境、工具打交道并不断纠偏。
- 推理扩展定律提醒我们：性能不仅取决于模型规模，还取决于系统愿意为推理投入多少时间与算力，从而换取更审慎、更高质量的行动。
- CoT 像智能体的自言自语，把宏大目标切成一串可执行的小动作。
- ToT 与自我纠正提供「多想几条路、走错了能退」的能力，在真正动手前先把方案打磨好。
- CoD 一类协作框架意味着系统正从单体智能体走向团队协作：多个智能体通过相互质询与交叉验证，共同处理更复杂的问题，并降低个体偏见。
- Deep Research 等产品证明，当上述技术堆到一块，智能体可以长时间、全自动地完成深度调研类任务。
- MASS 这类搜索框架则在系统层面回答「提示怎么写、拓扑怎么连」——用算法替人做 MAS 级别的超参搜索。
- 综合使用这些推理手段，才能从「会跑的脚本」进化到「少盯也能托付的自主智能体」。

Conclusions

结论

Modern AI is evolving from passive tools into autonomous agents, capable of tackling complex goals through structured reasoning. This agentic behavior begins with an internal monologue, powered by techniques like Chain-of-Thought (CoT), which allows an agent to formulate a coherent plan before acting. True autonomy requires deliberation, which agents achieve through Self-Correction and Tree-of-Thought (ToT), enabling them to evaluate multiple strategies and independently improve their own work. The pivotal leap to fully agentic systems comes from the ReAct framework, which empowers an agent to move beyond thinking and start acting by using external tools. This establishes the core agentic loop of thought, action, and observation, allowing the agent to dynamically adapt its strategy based on environmental feedback.

当代 AI 正在从「问一句答一句的工具」转向能够处理复杂目标的自主智能体。起点往往是 CoT 式的内心独白：先想清楚再动手。但要称得上自主，还需要会自我怀疑、会改方案——自我纠正与 ToT 让智能体在多种策略之间比较、试错、回滚。真正的分水岭则是 ReAct：不仅要想，还要能调用工具、读取环境，并在「思考—行动—观察」的闭环里实时改写计划。

An agent's capacity for deep deliberation is fueled by the Scaling Inference Law, where more computational "thinking time" directly translates into more robust autonomous actions. The next frontier is the multi-agent system, where frameworks like Chain of Debates (CoD) create collaborative agent societies that reason together to achieve a common goal. This is not theoretical; agentic applications like Deep Research already demonstrate how autonomous agents can execute complex, multi-step investigations on a user's behalf. The overarching goal is to engineer reliable and transparent autonomous agents that can be trusted to

independently manage and solve intricate problems. Ultimately, by combining explicit reasoning with the power to act, these methodologies are completing the transformation of AI into truly agentic problem-solvers.

更深层次的审慎推理，往往建立在推理扩展定律所允许的额外计算预算之上：增加的“思考时间”通常能够转化为更稳健的自主行为。再进一步，则是多智能体协作阶段，CoD 等框架让多个智能体以类似圆桌讨论的方式相互质证。Deep Research 已经证明，这种能力能够落地为代用户完成小型研究项目。工程上的终极目标，是构建可审计、可托付的自主系统：即使人类不持续在环监督，它也能将复杂问题拆解、执行并复盘。只有将显式推理与现实世界中的行动能力结合起来，AI 才能真正发展为面向复杂问题求解的智能体系统。

References

Relevant research includes:

1. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" by Wei et al. (2022)
2. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models" by Yao et al. (2023)
3. "Program-Aided Language Models" by Gao et al. (2023)
4. "ReAct: Synergizing Reasoning and Acting in Language Models" by Yao et al. (2023)
5. Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for LLM Problem-Solving, 2024
6. Multi-Agent Design: Optimizing Agents with Better Prompts and Topologies, https://arxiv.org/abs/2502.02533